

3b. Create and Test a Fabric User Data Functions

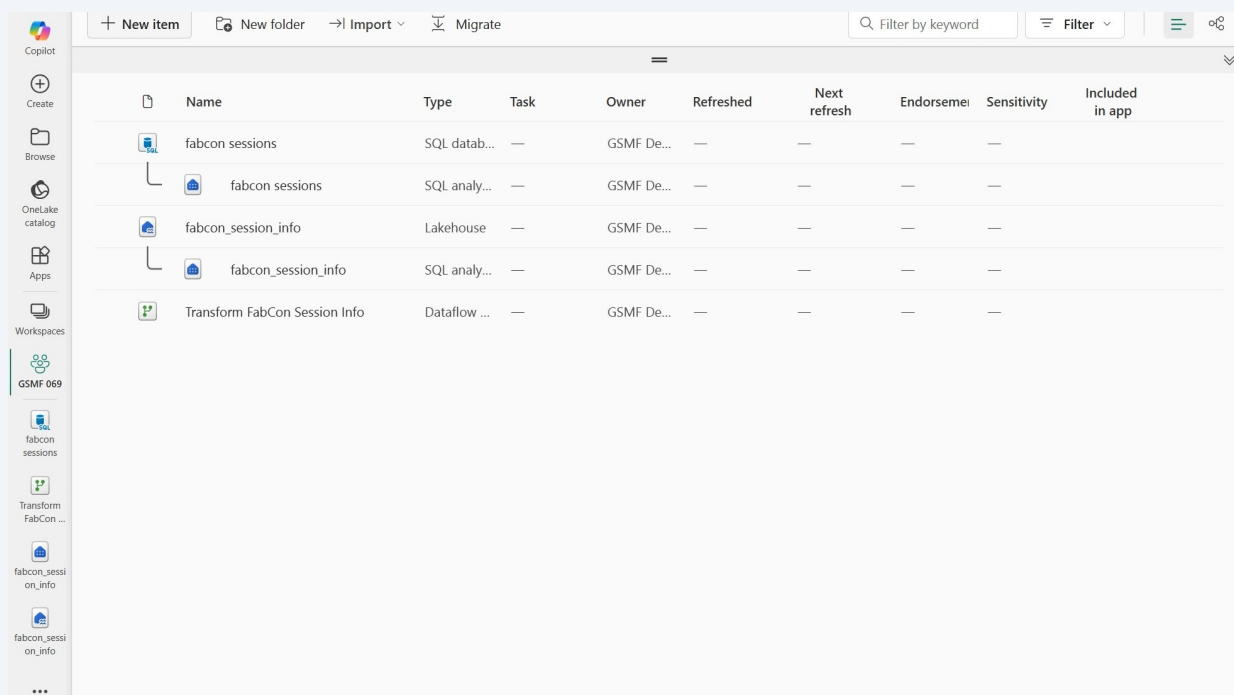
Fabric User Data Functions allow developers to write and host custom code inside Fabric. These functions can be invoked from Fabric Notebooks, Data Pipelines, or Power BI reports, and even accessed externally via REST endpoints.

We'll create two functions so we can write some data back to the SQL database from a Power BI report.

Create a new function

1

Navigate to your workspace. Notice how many items you've built so far! A lakehouse, and Dataflow Gen2 and a SQL database. Now we'll create a User Data Function, which we'll use to write data to our SQL database from a Power BI report.



The screenshot shows the Microsoft Fabric workspace interface. On the left is a sidebar with navigation options: Copilot, Create, Browse, OneLake catalog, Apps, Workspaces, and a list of items including 'GSMF 069', 'fabcon sessions', 'Transform FabCon ...', 'fabcon_sessi on_info', and 'fabcon_sessi on_info'. The main area displays a table of workspace items.

Name	Type	Task	Owner	Refreshed	Next refresh	Endorsemei	Sensitivity	Included in app
fabcon sessions	SQL datab...	—	GSMF De...	—	—	—	—	
fabcon sessions	SQL analy...	—	GSMF De...	—	—	—	—	
fabcon_session_info	Lakehouse	—	GSMF De...	—	—	—	—	
fabcon_session_info	SQL analy...	—	GSMF De...	—	—	—	—	
Transform FabCon Session Info	Dataflow ...	—	GSMF De...	—	—	—	—	

2 Click "New item"

The screenshot shows the Power BI workspace 'GSMF 069'. The left sidebar contains navigation icons for Home, Copilot, Create, Browse, OneLake catalog, Apps, and Workspaces. The main area displays a table of items. The 'New item' button is circled in orange in the top left of the main area.

Name	Type	Task	Owner	Refreshed	Next refresh	Endorsement	Sensitivity	Included in app
fabcon sessions	SQL datab...	—	GSMF De...	—	—	—	—	—
fabcon sessions	SQL analy...	—	GSMF De...	—	—	—	—	—
fabcon_session_info	Lakehouse	—	GSMF De...	—	—	—	—	—
fabcon_session_info	SQL analy...	—	GSMF De...	—	—	—	—	—
Transform FabCon Session Info	Dataflow ...	—	GSMF De...	—	—	—	—	—

3 Click the "Filter by item type" field and type "user" to easily locate the user data function.

The screenshot shows the 'New item' dialog box open in the Power BI workspace 'GSMF 069'. The dialog has tabs for 'Favorites' and 'All items'. The 'Filter by item type' field is circled in orange. Below the filter field, there are sections for 'Visualize data' and 'Get data'.

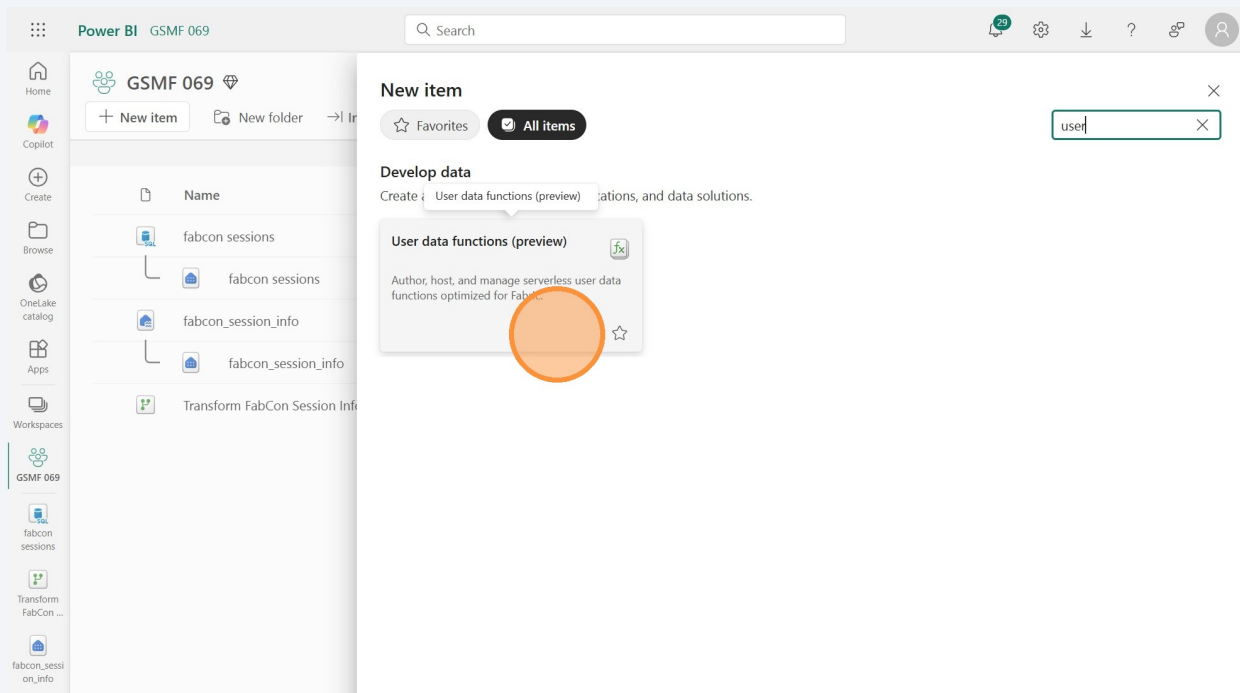
Visualize data

Present your data as rich visualizations and insights that can be shared with others.

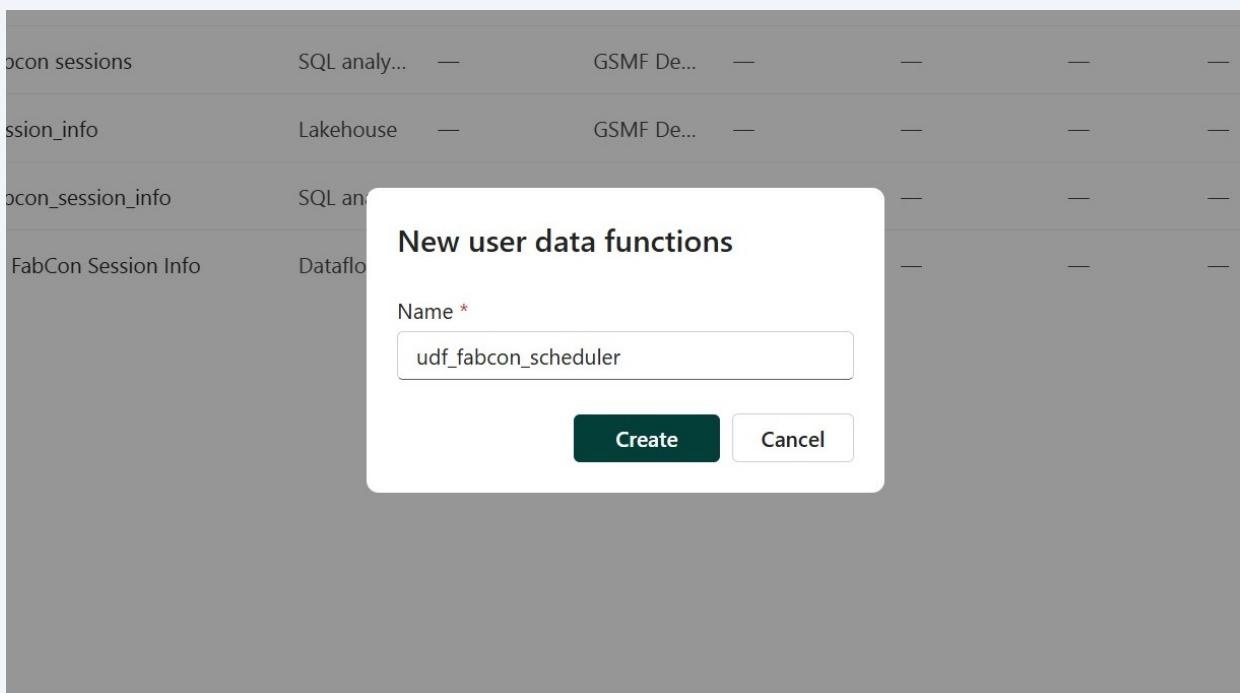
- Dashboard**: Build a single-page data story.
- Exploration (preview)**: Use lightweight tools to analyze your data and uncover trends.
- Org app (preview)**: Package and securely distribute content in your organization.
- Paginated Report (preview)**: Display tabular data in a report that's easy to print and share.
- Real-Time Dashboard**: Visualize key insights to share with your team.
- Report**: Create an interactive presentation of your data.
- Scorecard**: Define, track, and share key goals for your organization.

Get data

4 Click the "User data functions" tile.

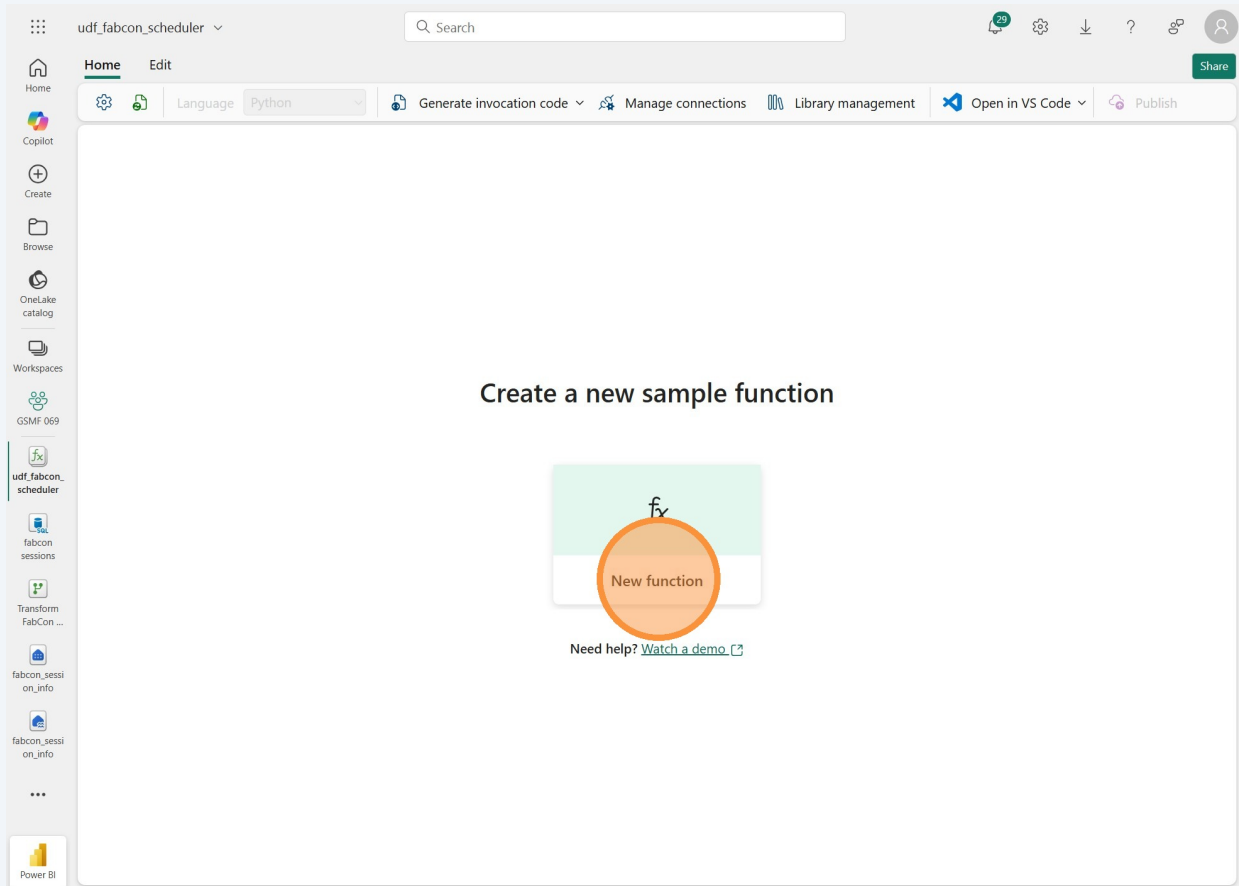


5 Give the function a name like "udf_fabcon_scheduler", for example, and click the "Create" button.



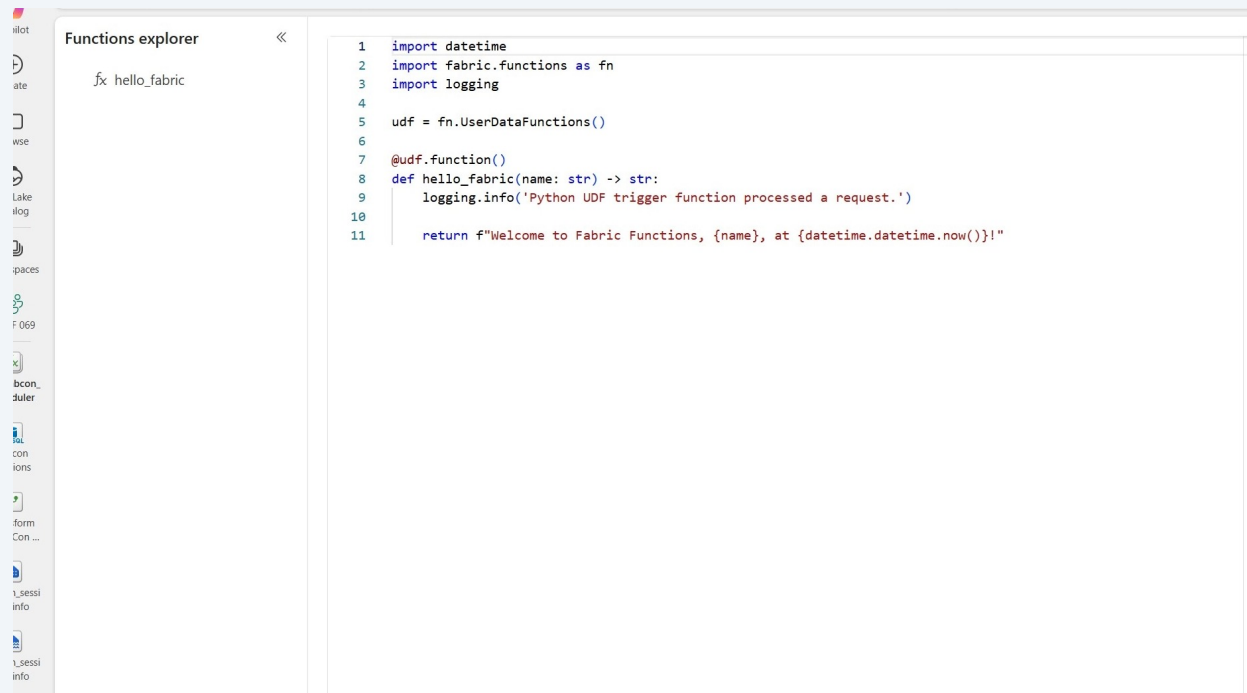
6

Now you're in the user data function editor. Click the big "New function" button in the middle of the canvas.



7

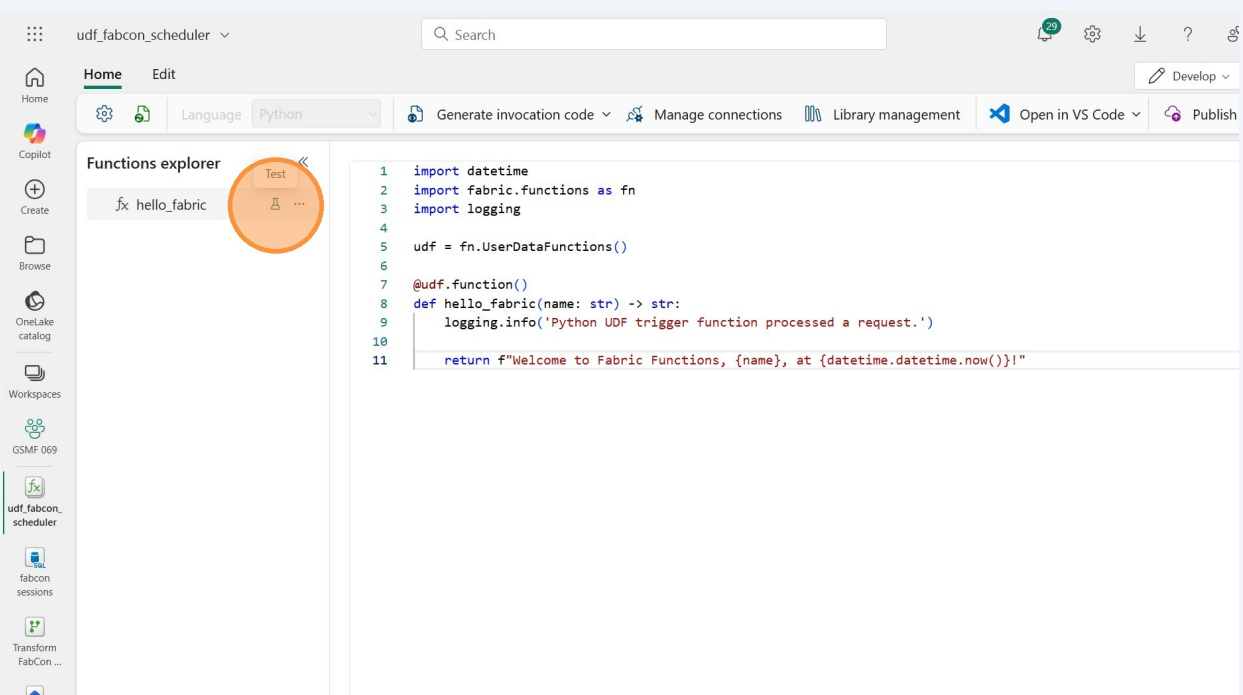
A sample function is automatically created. It doesn't do much other than return a message based on an input, but it gives you an idea of what the code should look like. We'll test this sample function to see what happens.



Test the sample function

8

Hover over the "hello_fabric" function and click the little test beaker to test the function.



9 Click the "Enter value" field and enter your name.

The screenshot shows the Azure Functions portal interface. On the left, the 'Functions explorer' shows a function named 'fx hello_fabric'. The main editor displays the Python code for the function:

```
1 import datetime
2 import fabric.functions as fn
3 import logging
4
5 udf = fn.UserDataFunctions()
6
7 @udf.function()
8 def hello_fabric(name: str) -> str:
9     logging.info('Python UDF trigger')
10
11     return f"Welcome to Fabric Funct"
```

On the right, the 'Test' tab is active. It shows the function name 'hello_fabric' and the parameters section. The 'Parameters' section has a 'Type: str' label and two input fields: 'name' and 'Enter value'. The 'Enter value' field is highlighted with an orange circle. Below the input fields is a 'Test' button. The 'Output (str)' and 'Logs' sections are empty.

10 Click the "Test" button. (make sure it gets clicked and doesn't just move your cursor)

The screenshot shows the same Azure Functions portal interface as the previous one. In the 'Test' tab, the 'name' input field now contains the text 'friends'. The 'Test' button is highlighted with an orange circle. The 'Output (str)' and 'Logs' sections remain empty.

11

After a few seconds you should see the output and any logging information. It may take a few seconds for it to kick off, so be patient.

The screenshot shows the Fabric Functions interface. On the left, the 'Functions explorer' pane lists 'fx hello_fabric'. The central editor displays the Python code for the function:

```
1 import datetime
2 import fabric.functions as fn
3 import logging
4
5 udf = fn.UserDataFunctions()
6
7 @udf.function()
8 def hello_fabric(name: str) -> str:
9     logging.info('Python UDF trigger')
10
11     return f'Welcome to Fabric Funct'
```

On the right, the 'Test' pane is open, showing the function name 'hello_fabric' and parameters 'name' and 'friends'. The status is 'Succeeded', and the output is 'Welcome to Fabric Functions, friends, at 2025-09-13 17:53:24.227268!'. The logs show 'Python UDF trigger function processed a request.'

12

Close the test pane by clicking the "X" in the top right.

The screenshot shows the Fabric Functions interface with the 'Test' pane closed. The 'Functions explorer' pane lists 'fx hello_fabric'. The central editor displays the Python code for the function:

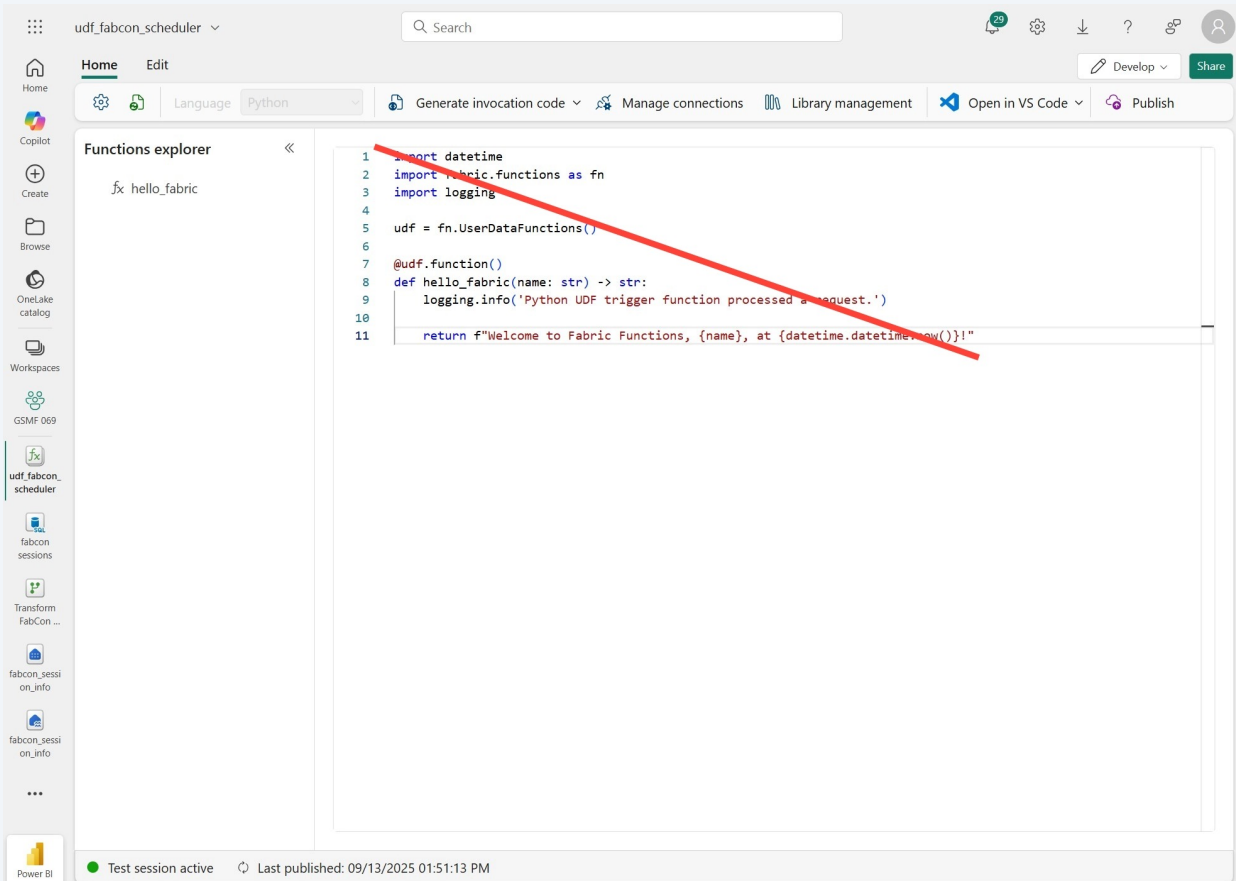
```
1 import datetime
2 import fabric.functions as fn
3 import logging
4
5 udf = fn.UserDataFunctions()
6
7 @udf.function()
8 def hello_fabric(name: str) -> str:
9     logging.info('Python UDF trigger')
10
11     return f'Welcome to Fabric Funct'
```

The 'Test' pane is no longer visible, and the 'X' button in the top right corner is highlighted with an orange circle.

Enter the code for the functions

13

Now we'll replace the sample function with a real function. Highlight all the code the delete it so there is no text left.

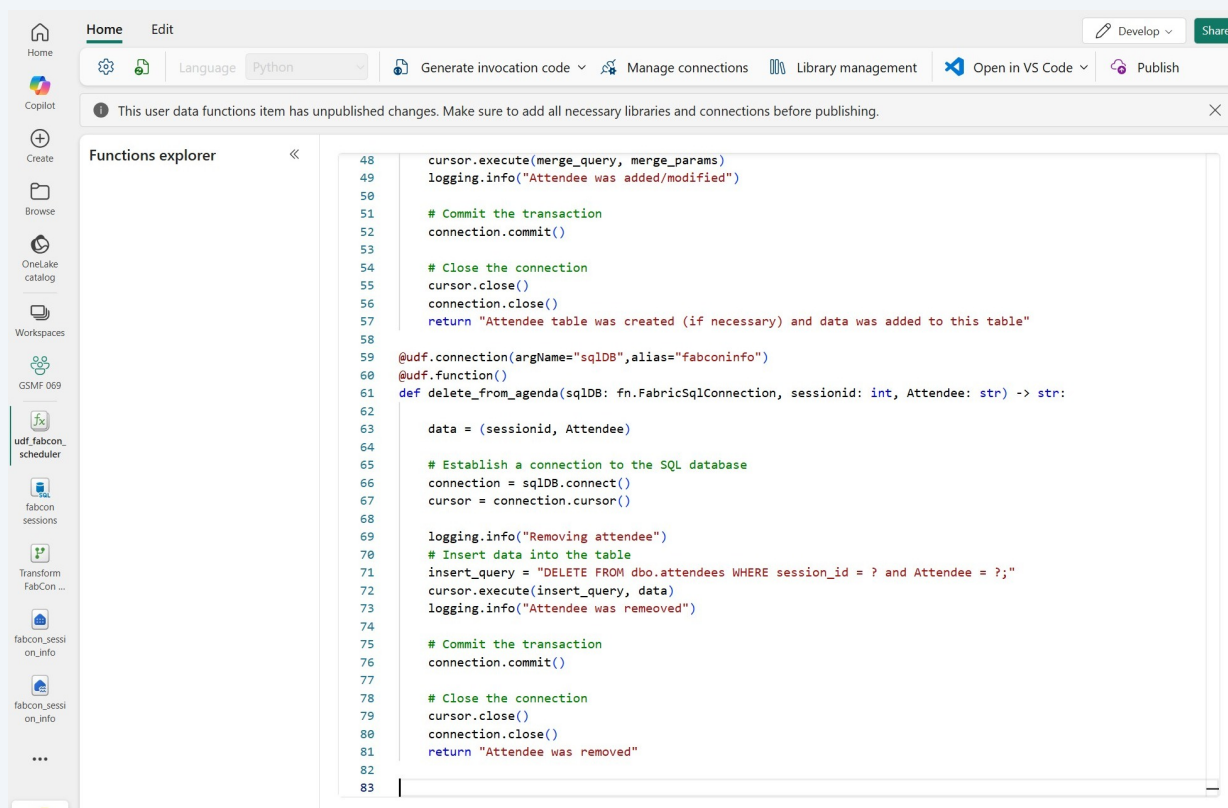


14

Open the UDF.txt file from the resources section and copy all the code. Paste it into the UDF editor.

15

Your function should now look like this. In fact, we have two functions (which you can see in the Functions explorer on the left). One for adding a session to your agenda, and one for deleting it from your agenda. Take a look at the code to get an idea of how it works.



Home Edit Develop Share

Language Python Generate invocation code Manage connections Library management Open in VS Code Publish

This user data functions item has unpublished changes. Make sure to add all necessary libraries and connections before publishing.

Functions explorer

```

48 cursor.execute(merge_query, merge_params)
49 logging.info("Attendee was added/modified")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created (if necessary) and data was added to this table"
58
59 @udf.connection(argName="sqlDB", alias="fabconinfo")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str) -> str:
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the SQL database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
70     # Insert data into the table
71     insert_query = "DELETE FROM dbo.attendees WHERE session_id = ? and Attendee = ?;"
72     cursor.execute(insert_query, data)
73     logging.info("Attendee was removed")
74
75     # Commit the transaction
76     connection.commit()
77
78     # Close the connection
79     cursor.close()
80     connection.close()
81     return "Attendee was removed"
82
83

```

udf fabcon_scheduler
fabcon_sessions
Transform FabCon ...
fabcon_sessi on_info
fabcon_sessi on_info

16

Notice there are two places where the text has a squiggly red line. You can see in the scrollbar on the right side that there are two little red squares indicating errors.

The screenshot shows the Microsoft Fabric IDE interface. The top bar includes a search bar and navigation icons. The left sidebar shows the 'Functions explorer' with a list of functions: 'fx save_to_agenda' and 'fx delete_from_agenda'. The main editor displays a Python script for a user-defined function named 'save_to_agenda'. The script imports 'fabric.functions as fn' and 'logging', then defines a function 'save_to_agenda' that takes 'sqlDB', 'sessionid', 'Attendee', and 'Status' as parameters. The function body includes comments and SQL queries to create a table and insert data. Two syntax errors are highlighted: a missing closing parenthesis in the '@udf.connection' decorator on line 6, and a missing closing parenthesis in the 'merge_query' string on line 28. The scrollbar on the right shows two red squares indicating these errors.

```
1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconinfo")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17     IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18     CREATE TABLE dbo.attendees (
19         [session_id] bigint NOT NULL,
20         [Attendee] NVARCHAR(250) NOT NULL,
21         [Status] NVARCHAR(50) NOT NULL
22     );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table
27     # insert_query = "INSERT INTO dbo.attendees (session_id, Attendee, Status) VALUES (?, ?, ?);"
28     merge_query = """
29     MERGE dbo.attendees AS target
30     USING (SELECT ? AS session_id, ? AS Attendee) AS src
31     ON target.session_id = src.session_id AND target.Attendee = src.Attendee
32     WHEN MATCHED THEN
33         UPDATE SET Status = ?
34     WHEN NOT MATCHED THEN
35         INSERT (session_id, Attendee, Status)
36         VALUES (?, ?, ?);
```

Test session active Last published: 09/13/2025 01:51:13 PM

17

If you click the error, you'll see a message telling you that we are referring to a connection that doesn't exist. We need to create a connection to our SQL database for this function to use as the connection.

udf_fabcon_scheduler ▾

Search

Home Edit

Language Python

Generate invocation code Manage connections Library management Open in VS Code Publish

This user data functions item has unpublished changes. Make sure to add all necessary libraries and connections before publishing.

Functions explorer

- fx save_to_agenda
- fx delete_from_agenda

```

1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconinfo")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17         IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18             CREATE TABLE dbo.attendees (
19                 [session_id] bigint NOT NULL,
20                 [Attendee] NVARCHAR(250) NOT NULL,
21                 [Status] NVARCHAR(50) NOT NULL
22             );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table

```

Connection alias "fabconinfo" is not added or updated in connections management, please add or update it before publishing. PythonIntellisenseProvider

View problem (Alt+F1) No quick fixes available

Create a connection to the SQL database

18

Click "Manage connections"

udf_fabcon_scheduler ▾

Search

Home Edit

Language Python

Generate invocation code Manage connections Library management Open in VS Code Publish

This user data functions item has unpublished changes. Make sure to add all necessary libraries and connections before publishing.

Functions explorer

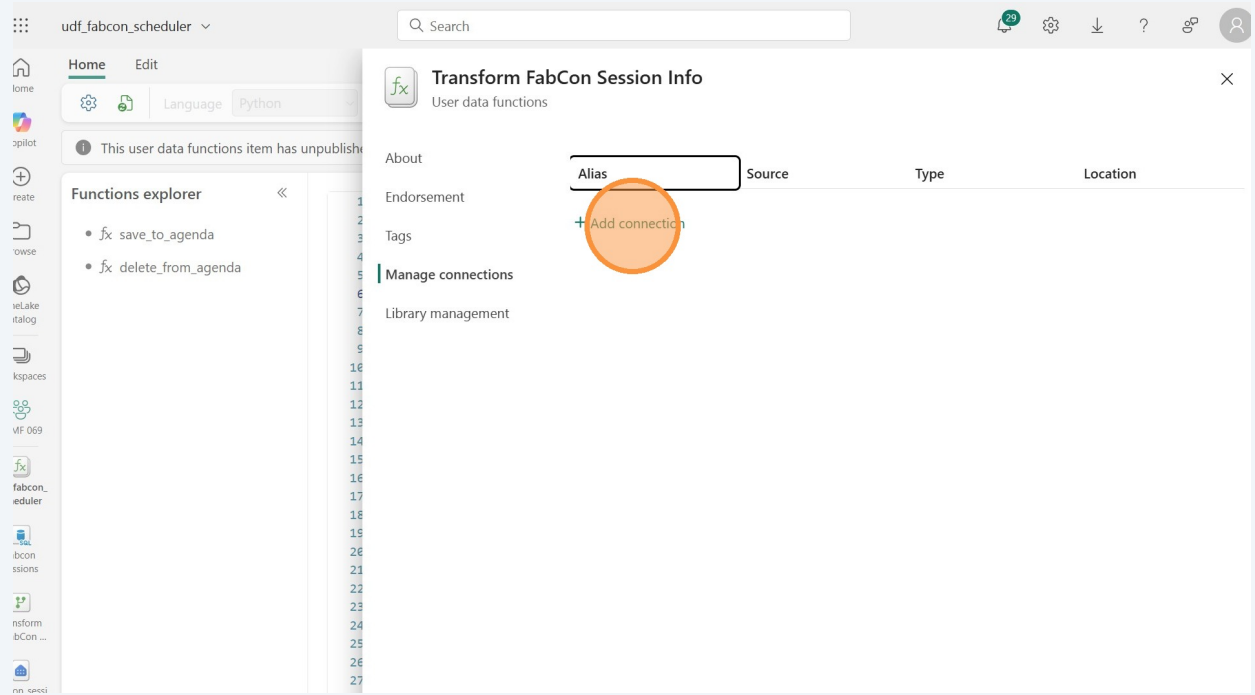
- fx save_to_agenda
- fx delete_from_agenda

```

1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconinfo")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17         IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18             CREATE TABLE dbo.attendees (
19                 [session_id] bigint NOT NULL,
20                 [Attendee] NVARCHAR(250) NOT NULL,
21                 [Status] NVARCHAR(50) NOT NULL
22             );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table

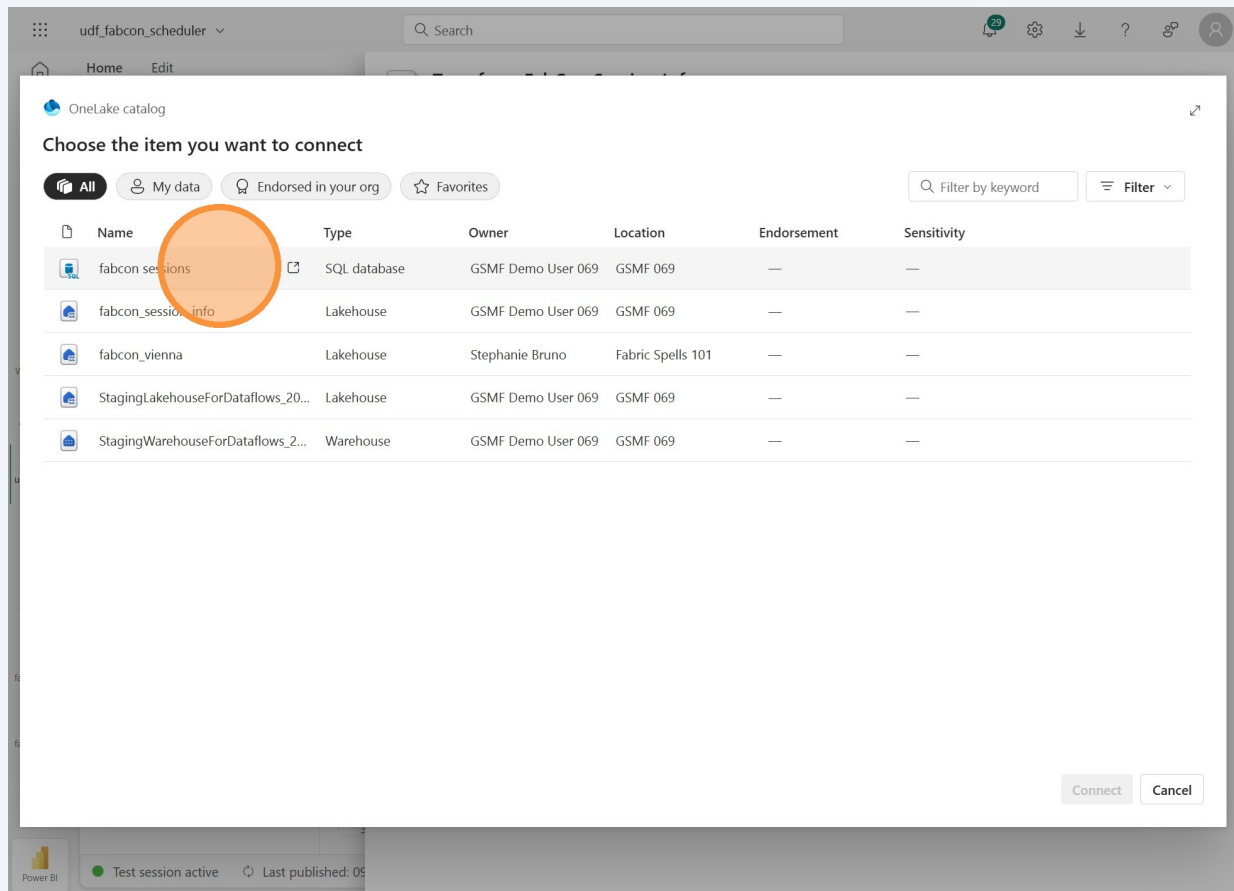
```

19 Click "Add connection"



20

Click on the SQL database you created in the previous lab. This is the one we want to connect to.



21 Click "Connect"

The screenshot shows the OneLake catalog interface. At the top, there's a search bar and navigation tabs. Below the tabs, a table lists various data items. The 'Connect' button is highlighted with an orange circle.

OneLake catalog

Choose the item you want to connect

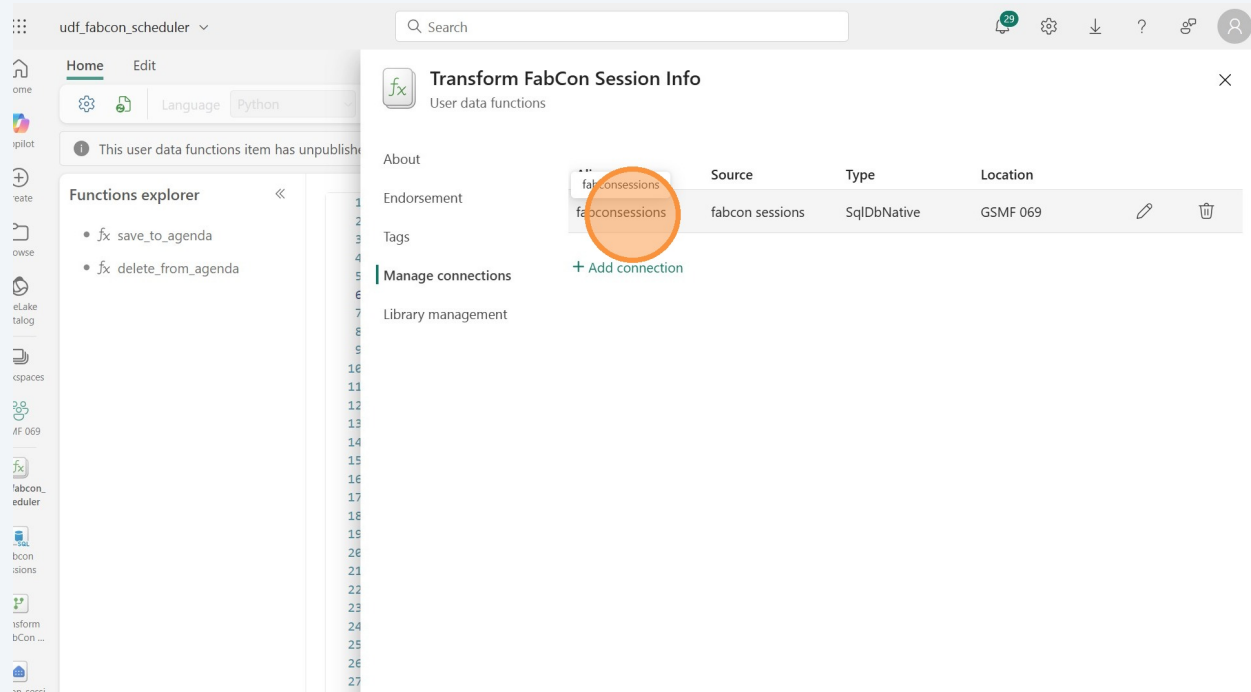
Filter by keyword Filter

Name	Type	Owner	Location	Endorsement	Sensitivity
fabcon sessions	SQL database	GSMF Demo User 069	GSMF 069	—	—
fabcon_session_info	Lakehouse	GSMF Demo User 069	GSMF 069	—	—
fabcon_vienna	Lakehouse	Stephanie Bruno	Fabric Spells 101	—	—
StagingLakehouseForDataflows_20...	Lakehouse	GSMF Demo User 069	GSMF 069	—	—
StagingWarehouseForDataflows_2...	Warehouse	GSMF Demo User 069	GSMF 069	—	—

Connect Cancel

22

You should now see one connection listed. The source is your SQL database and the type is SqlDbNative. The location is your workspace. It automatically created an alias for the connection based on your SQL database's name. You can either edit that alias or accept it as-is. Either way, the alias is what we need the code to refer to in the two lines using the connection. Copy the alias name so you can update the code with this value.



23 Click the "X" to close the connections pane.

The screenshot shows the Microsoft Fabric user interface. On the left is a navigation pane with icons for Home, Copilot, Create, Browse, OneLake catalog, Workspaces, GSMF 069, and a list of user data functions including 'udf_fabcon_scheduler'. The main area displays the 'Transform FabCon Session Info' pane, which is titled 'User data functions'. It contains a table with the following data:

Alias	Source	Type	Location
fabconsessions	fabcon sessions	SqlDbNative	GSMF 069

Below the table, there is a '+ Add connection' link and a 'Manage connections' section. An orange circle highlights the 'Close' button (an 'X' icon) in the top right corner of the 'Transform FabCon Session Info' pane. At the bottom of the interface, there is a status bar showing 'Test session active' and 'Last published: 09/...'.

24

Double-click "fabconinfo" on line 6 of the code and replace it with whatever your connection's alias name is.

udf_fabcon_scheduler

Search

Home Edit

Language Python

Generate invocation code Manage connections Library management Open in VS Code Publish

This user data functions item has unpublished changes. Make sure to add all necessary libraries and connections before publishing.

Functions explorer

- fx save_to_agenda
- fx delete_from_agenda

```

1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconinfo")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17         IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18             CREATE TABLE dbo.attendees (
19                 [session_id] bigint NOT NULL,
20                 [Attendee] NVARCHAR(250) NOT NULL,
21                 [Status] NVARCHAR(50) NOT NULL
22             );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table
  
```

Connection alias "fabconinfo" is not added or updated in connections management, please add or update it before publishing. PythonIntellisenseProvider

New Problem (Alt+F8) No quick fixes available

25

Do the same on line 59. After these two replacements, you shouldn't see the little red marks on the right-side scroll bar anymore.

```

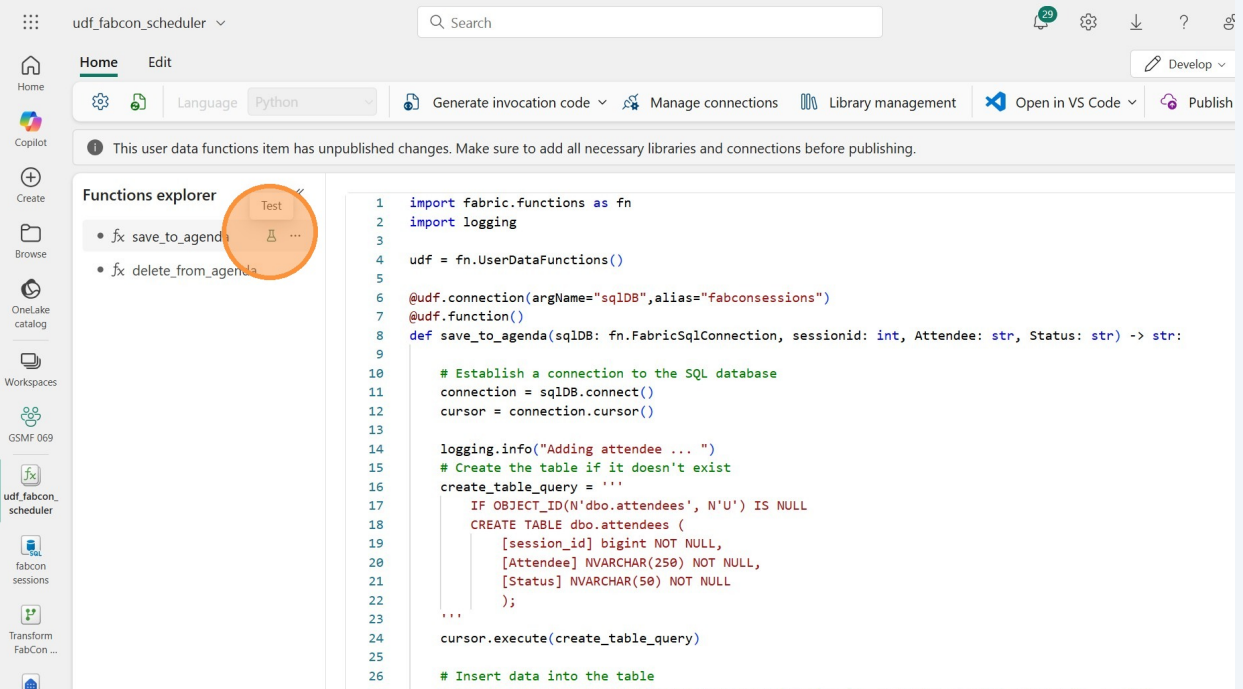
35 INSERT (session_id, Attendee, Status)
36 VALUES (?, ?, ?);
37
38
39 # Provide a value for each question mark above, in order:
40 merge_params = (
41     sessionid,      # SELECT ? AS session_id
42     Attendee,       # SELECT ? AS Attendee
43     Status,         # UPDATE SET [Status] = ?
44     sessionid,      # INSERT VALUES (?, ...
45     Attendee,       # ...
46     Status          # ...?)
47 )
48 cursor.execute(merge_query, merge_params)
49 logging.info("Attendee was added/modified")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created (if necessary) and data was added to this table"
58
59 @udf.connection(argName="sqlDB", alias="fabconinfo")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str) -> str:
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the SQL database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
  
```

Test session active Last published: 09/13/2025 01:51:13 PM

Test a function

26

Now we're ready to test! Click the "Test" icon next to the "save_to_agenda" function name.



The screenshot displays the Azure Data Studio interface for a project named 'udf.fabcon_scheduler'. The 'Functions explorer' on the left side shows two functions: 'fx save_to_agenda' and 'fx delete_from_agenda'. The 'Test' icon next to 'fx save_to_agenda' is highlighted with an orange circle. The main editor window shows the Python code for the 'save_to_agenda' function, which is a user-defined function (UDF) that connects to a SQL database, creates a table if it doesn't exist, and inserts data into it.

```
1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconsessions")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17         IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18             CREATE TABLE dbo.attendees (
19                 [session_id] bigint NOT NULL,
20                 [Attendee] NVARCHAR(250) NOT NULL,
21                 [Status] NVARCHAR(50) NOT NULL
22             );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table
```

27

The "save_to_agenda" takes three parameters: sessionid, Attendee, and Status. For testing purposes, it doesn't really matter what you enter, but sessionid must be an integer.

Enter values for each of the three parameters and click the "Test" button.

The screenshot shows the Azure Data Studio interface with the 'Test' dialog open for the 'save_to_agenda' function. The dialog is titled 'Test' and contains the following sections:

- Function name:** A dropdown menu showing 'save_to_agenda'.
- Parameters:** Three input fields with labels and types:
 - Type: int, sessionid: 10
 - Type: str, Attendee: me me me
 - Type: str, Status: Definitely
- Test button:** A green button with the text 'Test' is highlighted with an orange circle.
- Output (SQL):** A text area for the output of the function.
- Logs:** A text area for the logs of the function.

The background shows the 'Functions explorer' on the left with a list of functions: 'fx save_to_agenda' and 'fx delete_from_agenda'. The central editor displays the Python code for the 'save_to_agenda' function, which includes imports for 'fabric.functions' and 'logging', and a 'def save_to_agenda' function that establishes a connection to a SQL database and executes a query to insert data into a table named 'attendees'.

28

While the function is running you can see the status is "In progress."

The screenshot displays the Azure Data Studio interface with a Python function named `save_to_agenda` being tested. The interface includes a sidebar with a 'Functions explorer' showing the function, a central code editor with the function's source code, and a 'Test' panel on the right.

Functions explorer: Lists the function `fx save_to_agenda` and `fx delete_from_agenda`.

Code Editor: Contains the following Python code:

```
1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="sqlDB")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSQLDatabase):
9
10     # Establish a connection to the database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ..")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17     IF OBJECT_ID(N'dbo.attendees') IS NULL
18     CREATE TABLE dbo.attendees (
19         [session_id] bigint NOT NULL,
20         [Attendee] NVARCHAR(250) NOT NULL,
21         [Status] NVARCHAR(50) NOT NULL
22     );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table
27     # insert_query = "INSERT INTO dbo.attendees (session_id, Attendee, Status) VALUES (?, ?, ?)"
28     merge_query = """
29     MERGE dbo.attendees AS target
30     USING (SELECT ? AS session_id, ? AS Attendee, ? AS Status) AS source
31     ON target.session_id = source.session_id
32     WHEN MATCHED THEN
33         UPDATE SET Status = source.Status
34     WHEN NOT MATCHED THEN
35         INSERT (session_id, Attendee, Status)
36         VALUES (?, ?, ?);
37     """
38     cursor.execute(merge_query)
```

Test Panel: Shows the function name `save_to_agenda` and parameters: `sessionid` (10), `Attendee` (me me me), and `Status` (Definitely). The status is **In progress**, highlighted with an orange box.

Output (str): Empty text area for the function's output.

Logs: Empty text area for the function's logs.

Footer: Test session active. Last published: 09/13/2025 01:51:13 PM.

29

After it's finished you can see the output and logs again. We can see that we added some data to the attendee table.

The screenshot displays the Azure Data Studio interface for a user-defined function (UDF) named `udf_fabcon_scheduler`. The interface is divided into three main sections:

- Functions explorer:** Lists the functions `fx_save_to_agenda` and `fx_delete_from_agenda`.
- Code editor:** Contains the Python and SQL code for the UDF. The code imports `fabric.functions` and `logging`, establishes a connection to the SQL database, and executes a query to create the `dbo.attendees` table if it doesn't exist. It then inserts data into the table using a `MERGE` statement.
- Test panel:** Shows the parameters for the function: `sessionid` (int, 10), `Attendee` (str, me me me), and `Status` (str, Definitely). The test result is "Succeeded". The output message is "Attendee table was created (if necessary) and data was added to this table". The logs show "Adding attendee ..." and "Attendee was added/modified".

Verify the data in the SQL database

30

Let's go see the data in the SQL database. Go back to the SQL database by clicking on its icon on the left, or by going back to the workspace and clicking on it there.

The screenshot shows the Microsoft Fabric interface. On the left, the 'Functions Explorer' pane displays a list of functions, including 'fx_save_to_agenda' and 'fx_delete_from_agenda'. A red circle highlights the 'fabcon_sessions' function. The main pane shows a Python script for a user-defined function (UDF) named 'save_to_agenda'. The script imports 'fabric.functions as fn' and 'logging', and defines a function 'def save_to_agenda(sqlDB: fn.FabricSQLDatabase)'. The script includes logic to establish a connection to the SQL database, create a table named 'attendees' if it doesn't exist, and insert data into the table using a MERGE statement. The 'Test' dialog box is open, showing parameters: 'sessionid' (int, 10), 'Attendee' (str, 'me me me'), and 'Status' (str, 'Definitely'). The 'Test' button is highlighted. The 'Status' section shows 'Succeeded' with a green checkmark. The 'Output (str)' section shows 'Attendee table was created (if necessary) and data was added to this table'. The 'Logs' section shows 'Adding attendee ...' and 'Attendee was added/modified'.

31

Click on the attendees table to see a preview of the data. If you don't see anything yet, click on the refresh button in the Home ribbon or next to the Tables folder. It's possible you might still not see it in the preview (sometimes it takes a few minutes to show changes).

The screenshot shows the Microsoft Fabric interface. The 'Home' ribbon is active, and the 'Refresh' button (a circular arrow icon) is highlighted with a red circle. The 'Explorer' pane on the left shows the 'fabcon_sessions' workspace expanded, with the 'dbo' folder expanded, and the 'Tables' folder expanded. The 'attendees' table is selected, and its data preview is shown in the main pane. The data preview shows a table with two columns: 'session_id' and 'Attendee'. The table contains one row of data: 'ABC session_id' and 'ABC Attendee'.

32

If the preview doesn't show anything yet, you can see it by writing a SQL query. Click "New Query."

The screenshot shows the Microsoft Fabric interface. On the left is a sidebar with navigation icons for Home, Copilot, Create, Browse, OneLake catalog, Workspaces, GSMF 069, and udf_fabcon_scheduler. The main area has a top bar with 'fabcon sessions' and a search box. Below this is a tabbed interface with 'Home', 'Replication', and 'Security'. The 'Home' tab is active, showing a toolbar with icons for 'Get data', 'New Query' (highlighted with an orange circle), 'Templates', 'Open in', 'New API for GraphQL', and 'Performance summary'. The 'Explorer' pane on the left shows a tree view with 'fabcon sessions' expanded, containing 'dbo', 'Tables', 'attendees', 'Views', 'Stored Procedure...', 'Functions', 'Queries', and 'My queries'. The 'attendees' table is selected. The main pane shows a 'Data preview - attendees' table with columns 'session_id' and 'Attendee'.

33

Enter "SELECT * FROM attendees" and then click the "Run" button.

The screenshot shows the Microsoft Fabric interface with a SQL query entered and the 'Run' button highlighted. The 'New Query' button from the previous screenshot is now a tab labeled 'SQL query 1'. The 'Run' button (a play icon) is highlighted with an orange circle. The main pane shows the SQL query: `1 SELECT * FROM attendees`. Above the query editor are buttons for 'Save as view', 'Explain query', and 'Fix query errors'. A 'Preview' button is also visible. A warning message states: 'Copilot uses AI. Mistakes can happen. Verify code suggestions before running them. [Review](#)'. The 'Explorer' pane on the left remains the same, with 'attendees' selected.

34 You should now see one row. You populated the table by running the UDF!

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Explorer pane displays the database structure, including the 'dbo' schema, 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'Queries'. The 'Queries' folder is expanded, showing 'My queries' and 'SQL query 1' and 'SQL query 2'. The 'Results' tab is selected, showing the output of the query 'SELECT * FROM attendees'. The results are displayed in a table with three columns: 'session_id', 'Attendee', and 'Status'. A single row is shown with the values '10', 'me me me', and 'Definitely'.

ABC session_id	ABC Attendee	ABC Status
10	me me me	Definitely

35 Now go back to the UDF so we can test the delete function. You'll need to use the same session_id and Attendee so it can delete the correct row.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Explorer pane displays the database structure, including the 'dbo' schema, 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'Queries'. The 'Queries' folder is expanded, showing 'My queries' and 'SQL query 1' and 'SQL query 2'. The 'Results' tab is selected, showing the output of the query 'SELECT * FROM attendees'. The results are displayed in a table with three columns: 'session_id', 'Attendee', and 'Status'. A single row is shown with the values '10', 'me me me', and 'Definitely'. The 'udf_fabcon_scheduler' function is highlighted in the Explorer pane.

ABC session_id	ABC Attendee	ABC Status
10	me me me	Definitely

Test the second function

36

If you didn't close the test window, you can simply scroll to the top and select the delete function in the Function name dropdown. This should leave in the parameters you already had. If you did close it, just hover over the delete function and click the test tube to test. Then fill in the same sessionid and Attendee values you used for the first test and click the "Test" button.

The screenshot shows the Azure Data Studio interface with the 'Test' window open for the 'delete_from_agenda' function. The 'Function name' dropdown is set to 'delete_from_agenda'. The 'Parameters' section shows 'sessionid' with a value of '10' and 'Attendee' with a value of 'me me me'. The 'Test' button is highlighted with an orange circle. The 'Output (str)' and 'Logs' sections are empty. The background shows the 'Functions explorer' with 'fx delete_from_agenda' selected and the code editor displaying the function's implementation.

```
44 sessionid, # IN
45 Attendee, # ..
46 Status # ..
47 )
48 cursor.execute(merge_query, merge_data)
49 logging.info("Attendee was added")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created"
58
59 @udf.connection(argName="sqlDB", alias="sqlDB")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricConnection):
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
70     # Insert data into the table
71     insert_query = "DELETE FROM dbo.Attendee WHERE sessionid = %s"
72     cursor.execute(insert_query, data)
73     logging.info("Attendee was removed")
74
75     # Commit the transaction
76     connection.commit()
77
78     # Close the connection
79     cursor.close()
80     connection.close()
```

37 The output and logs should tell you that you deleted a row.

The screenshot displays the Azure Data Studio interface with a Python function named `delete_from_agenda` being tested. The function is located in the `udf_fabcon_scheduler` workspace. The code in the editor includes comments and SQL queries for deleting an attendee from a table. The `Test` panel on the right shows the function name, parameters (sessionid: 10, Attendee: me me me), and the output 'Attendee was removed'. The logs show 'Removing attendee' and 'Attendee was removed'.

```
44 sessionid, # IN
45 Attendee, # ..
46 Status # ..
47 )
48 cursor.execute(merge_query, merge_data)
49 logging.info("Attendee was added")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created"
58
59 @udf.connection(argName="sqlDB", alias="sqlDB")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabconSession):
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
70     # Insert data into the table
71     insert_query = "DELETE FROM dbo.Attendee WHERE sessionid = %s AND Attendee = %s"
72     cursor.execute(insert_query, data)
73     logging.info("Attendee was removed")
74
75     # Commit the transaction
76     connection.commit()
77
78     # Close the connection
79     cursor.close()
80     connection.close()
```

Test

delete_from_agenda

Parameters

Type: int sessionid 10

Type: str Attendee me me me

Test

Status

✓ Succeeded

Output (str)

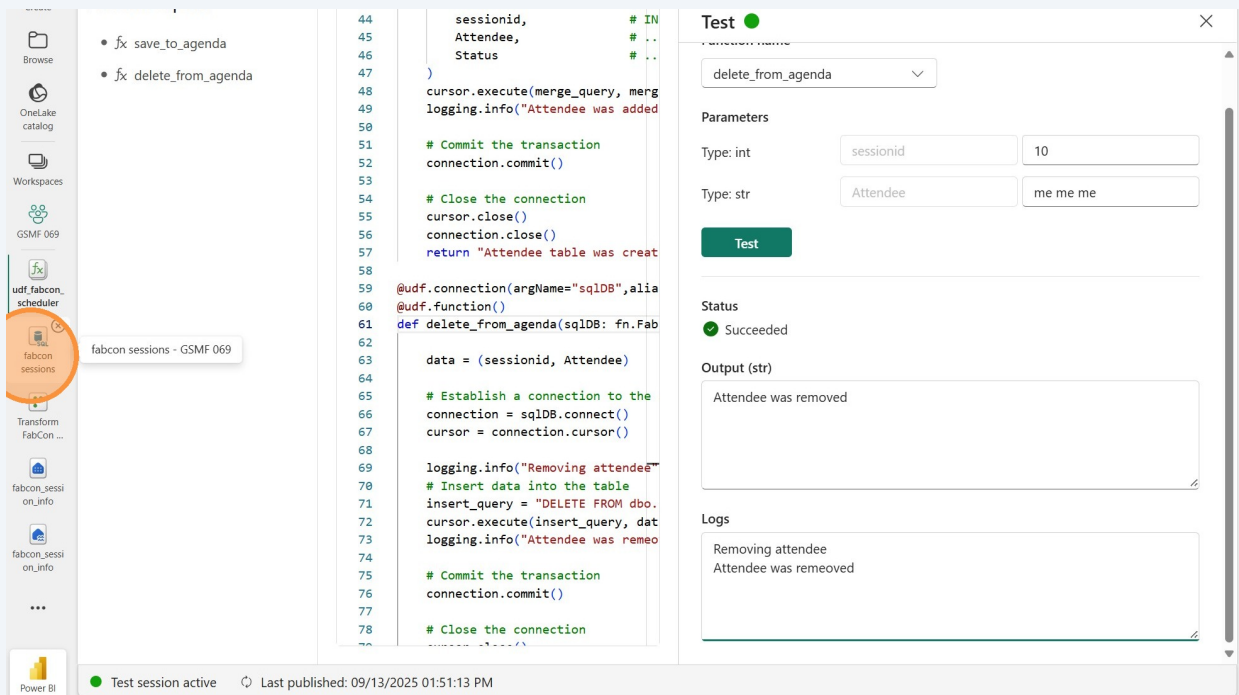
Attendee was removed

Logs

Removing attendee
Attendee was removed

Test session active Last published: 09/13/2025 01:51:13 PM

38 Go back to the SQL database again and we'll make sure the row is gone.



```
44 sessionid, # IN
45 Attendee, # ..
46 Status # ..
47 )
48 cursor.execute(merge_query, merge_data)
49 logging.info("Attendee was added")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created"
58
59 @udf.connection(argName="sqlDB", alias="sqlDB")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricConnection):
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
70     # Insert data into the table
71     insert_query = "DELETE FROM dbo.attendees WHERE session_id = %s AND attendee = %s"
72     cursor.execute(insert_query, data)
73     logging.info("Attendee was removed")
74
75     # Commit the transaction
76     connection.commit()
77
78     # Close the connection
79     cursor.close()
80     connection.close()
81     return "Attendee was removed"
```

Test

delete_from_agenda

Parameters

Type: int sessionid 10

Type: str Attendee me me me

Test

Status

✓ Succeeded

Output (str)

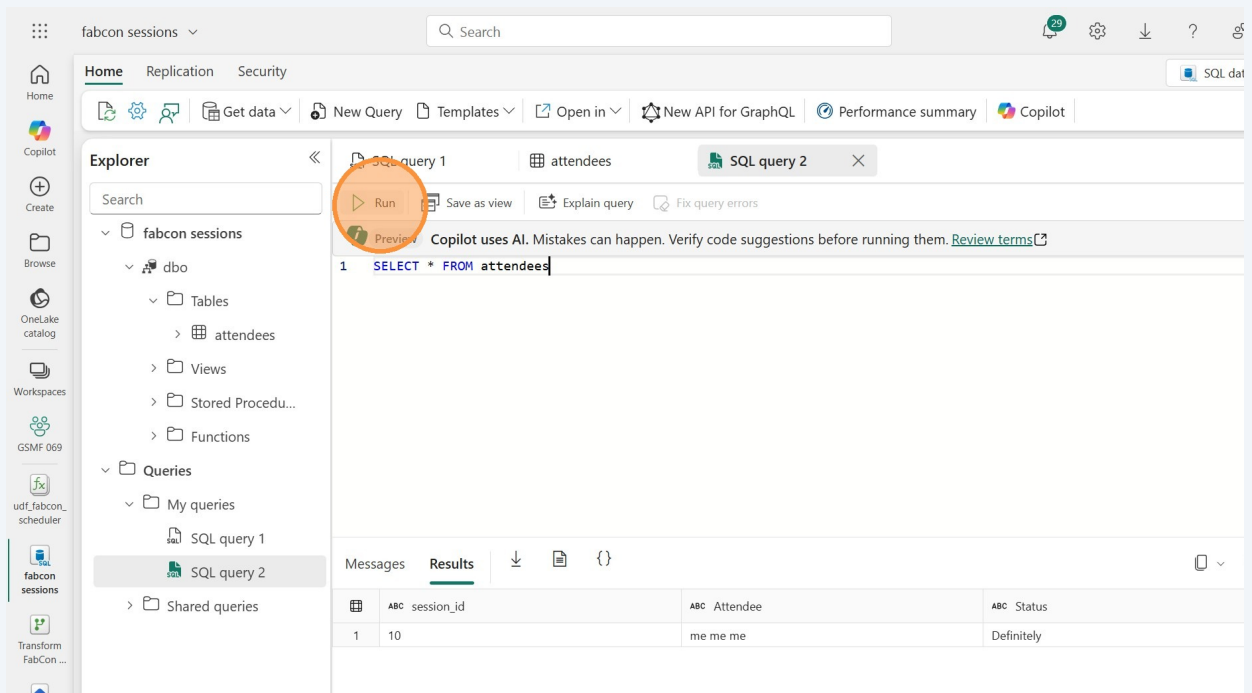
Attendee was removed

Logs

Removing attendee
Attendee was removed

Test session active Last published: 09/13/2025 01:51:13 PM

39 Rerun the SQL query.



fabcon sessions

Search

Home Replication Security

Get data New Query Templates Open in New API for GraphQL Performance summary Copilot

Explorer

fabcon sessions

dbo

Tables

attendees

Views

Stored Procedures

Functions

Queries

My queries

SQL query 1

SQL query 2

Shared queries

SQL query 1 attendees SQL query 2

Run Save as view Explain query Fix query errors

Copilot uses AI. Mistakes can happen. Verify code suggestions before running them. [Review terms](#)

```
1 SELECT * FROM attendees
```

Messages Results

	ABC session_id	ABC Attendee	ABC Status
1	10	me me me	Definitely

40

The row is gone! Back to the empty table. Navigate back to the UDF.

The screenshot shows the Microsoft Fabric SQL interface. On the left, the Explorer pane displays the database structure for 'fabcon sessions', including 'dbo', 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'Queries'. The 'udf_fabcon_scheduler' is highlighted under 'Queries'. The main pane shows 'SQL query 2' with the query 'SELECT * FROM attendees'. The 'Results' tab is active, displaying a table with three columns: 'session_id', 'Attendee', and 'Status'. The table is empty, with 0 rows. The status bar at the bottom indicates 'Succeeded (0 sec 698 ms)' and 'Copilot completions: On'.

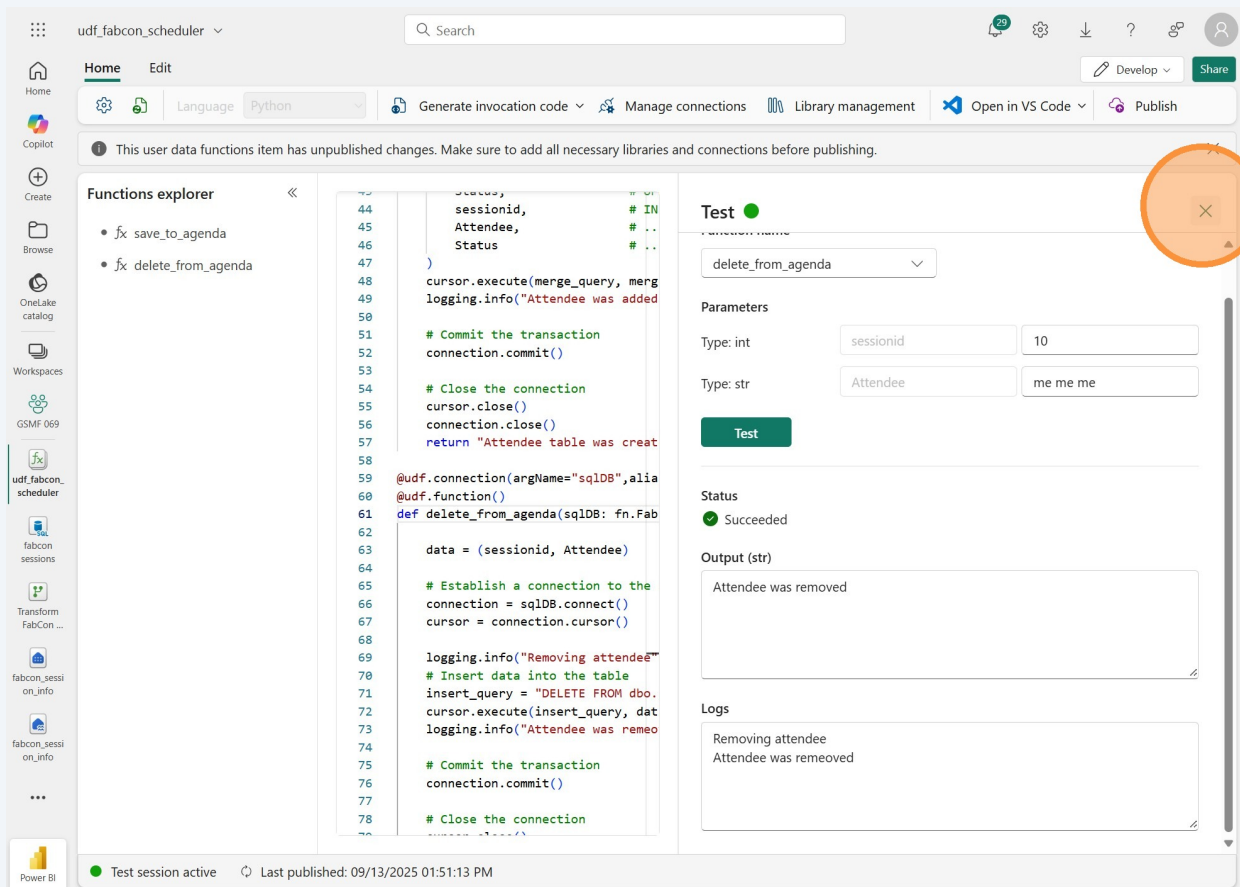
SQL query 2

```
SELECT * FROM attendees
```

session_id	Attendee	Status
------------	----------	--------

Succeeded (0 sec 698 ms) Copilot completions: On Copilot completions: Ready Columns: 3 Rows: 0

41 Close the test pane if you've still got it open.



The screenshot shows the Azure Data Studio interface with the 'Test' pane open on the right. The 'Test' pane displays the function name 'delete_from_agenda', parameters for 'sessionid' (10) and 'Attendee' (me me me), and a status of 'Succeeded'. The output shows 'Attendee was removed'. The logs show 'Removing attendee' and 'Attendee was removed'. An orange circle highlights the close button (X) in the top right corner of the 'Test' pane. The main editor shows the Python code for the 'delete_from_agenda' function. The left sidebar shows the 'Functions explorer' with 'fx delete_from_agenda' selected. The bottom status bar indicates 'Test session active' and 'Last published: 09/13/2025 01:51:13 PM'.

```
44 sessionid, # IN
45 Attendee, # ..
46 Status # ..
47 )
48 cursor.execute(merge_query, merge_data)
49 logging.info("Attendee was added")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created"
58
59 @udf.connection(argName="sqlDB", alias="sqlDB")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricConnection):
62
63     data = (sessionid, Attendee)
64
65     # Establish a connection to the database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
70     # Insert data into the table
71     insert_query = "DELETE FROM dbo.Attendee WHERE sessionid = %s AND Attendee = %s"
72     cursor.execute(insert_query, data)
73     logging.info("Attendee was removed")
74
75     # Commit the transaction
76     connection.commit()
77
78     # Close the connection
79     cursor.close()
80     connection.close()
```

Publish the User data function

42

Click "Publish" to publish the function. After your functions are published, other users and Fabric items can run your functions.

The screenshot shows the Azure Data Studio interface for a user-defined function (UDF) named 'udf_fabcon_scheduler'. The top bar includes a search bar, a 'Publish' button (highlighted with an orange circle), and a 'Share' button. Below the top bar, there is a notification bar stating: 'This user data functions item has unpublished changes. Make sure to add all necessary libraries and connections before publishing.' The main editor area displays the Python code for the UDF. The code includes a function definition for 'delete_from_agenda' that takes 'sqlDB' as an argument and 'sessionId' and 'Attendee' as parameters. The function logic involves establishing a connection to the SQL database, executing a query to remove the attendee, and logging the action. The 'Functions explorer' on the left side shows two functions: 'fx_save_to_agenda' and 'fx_delete_from_agenda'.

```
44 sessionid,          # INSERT VALUES (?, ...
45 Attendee,          # ...
46 Status             # ...?)
47 )
48 cursor.execute(merge_query, merge_params)
49 logging.info("Attendee was added/modified")
50
51 # Commit the transaction
52 connection.commit()
53
54 # Close the connection
55 cursor.close()
56 connection.close()
57 return "Attendee table was created (if necessary) and data was added to this table"
58
59 @udf.connection(argName="sqlDB", alias="fabconsessions")
60 @udf.function()
61 def delete_from_agenda(sqlDB: fn.FabricSqlConnection, sessionId: int, Attendee: str) -> str:
62
63     data = (sessionId, Attendee)
64
65     # Establish a connection to the SQL database
66     connection = sqlDB.connect()
67     cursor = connection.cursor()
68
69     logging.info("Removing attendee")
```

43 You'll see a message that the function is being published.

The screenshot displays the Azure Data Studio interface with the 'udf_fabcon_scheduler' project open. The 'Functions explorer' on the left lists two functions: 'fx save_to_agenda' and 'fx delete_from_agenda'. The main editor shows the Python code for the 'udf_fabcon_scheduler' function, which includes imports for 'fabric.functions' and 'logging', and a 'def save_to_agenda' function that connects to a SQL database and inserts data into a table named 'dbo.attendees'.

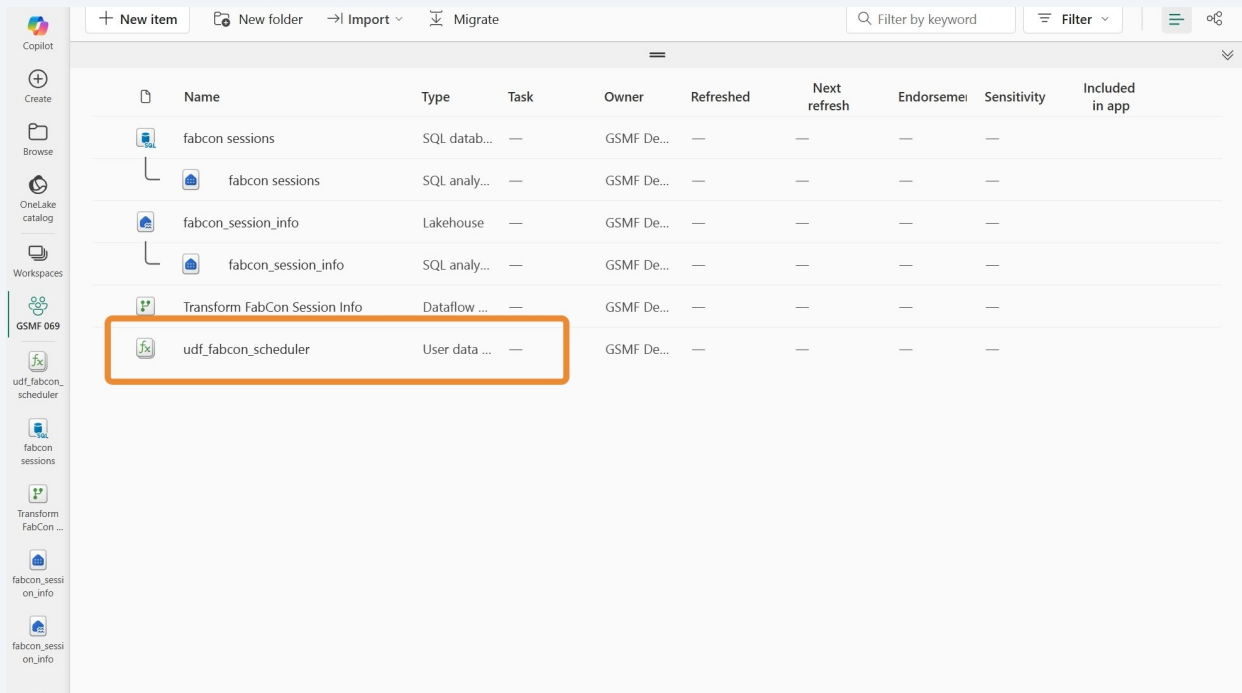
A notification box in the top right corner, titled 'Publishing', states: 'udf_fabcon_scheduler is being published. User data functions'. This box is highlighted with an orange border.

The bottom status bar shows two indicators: 'Test session active' (green dot) and 'Publishing in progress' (green circle).

```
1 import fabric.functions as fn
2 import logging
3
4 udf = fn.UserDataFunctions()
5
6 @udf.connection(argName="sqlDB", alias="fabconsessions")
7 @udf.function()
8 def save_to_agenda(sqlDB: fn.FabricSqlConnection, sessionid: int, Attendee: str, Status: str) -> str:
9
10     # Establish a connection to the SQL database
11     connection = sqlDB.connect()
12     cursor = connection.cursor()
13
14     logging.info("Adding attendee ... ")
15     # Create the table if it doesn't exist
16     create_table_query = '''
17         IF OBJECT_ID(N'dbo.attendees', N'U') IS NULL
18             CREATE TABLE dbo.attendees (
19                 [session_id] bigint NOT NULL,
20                 [Attendee] NVARCHAR(250) NOT NULL,
21                 [Status] NVARCHAR(50) NOT NULL
22             );
23     '''
24     cursor.execute(create_table_query)
25
26     # Insert data into the table
27     # insert_query = "INSERT INTO dbo.attendees (session_id, Attendee, Status) VALUES (?, ?, ?);"
28     merge_query = """
29         MERGE dbo.attendees AS target
30         USING (SELECT ? AS session_id, ? AS Attendee) AS src
31         ON target.session_id = src.session_id AND target.Attendee = src.Attendee
32         WHEN MATCHED THEN
33             UPDATE SET Status = ?
34         WHEN NOT MATCHED THEN
35             INSERT (session_id, Attendee, Status)
36             VALUES (?, ?, ?);
37     """
38
```

44

Go back to the workspace and you'll see that you now have a User Data Function.



The screenshot shows the Microsoft Fabric workspace interface. On the left is a sidebar with navigation options: Copilot, Create, Browse, OneLake catalog, Workspaces, and a list of items including 'GSMF 069', 'udf_fabcon_scheduler', 'fabcon sessions', 'Transform FabCon ...', 'fabcon_sessi on_info', and 'fabcon_sessi on_info'. The main area displays a table of items. The table has columns: Name, Type, Task, Owner, Refreshed, Next refresh, Endorsement, Sensitivity, and Included in app. The item 'udf_fabcon_scheduler' is highlighted with an orange box.

Name	Type	Task	Owner	Refreshed	Next refresh	Endorsement	Sensitivity	Included in app
fabcon sessions	SQL datab...	—	GSMF De...	—	—	—	—	
fabcon sessions	SQL analy...	—	GSMF De...	—	—	—	—	
fabcon_session_info	Lakehouse	—	GSMF De...	—	—	—	—	
fabcon_session_info	SQL analy...	—	GSMF De...	—	—	—	—	
Transform FabCon Session Info	Dataflow ...	—	GSMF De...	—	—	—	—	
udf_fabcon_scheduler	User data ...	—	GSMF De...	—	—	—	—	